3F7 Information Theory and Coding
Full Technical Report

Liu Zihe
zl559

University of Cambridge
December 6 2025

1 Introduction

Information theory provides the mathematical framework for quantifying information and establishing fundamental limits on data compression. Shannon's Source Coding Theorem establishes the entropy $H(X)$ as the lower bound for the average code length of an independent and identically distributed (i.i.d.) source. While i.i.d. models are useful first approximations, real-world data sources—such as natural language—exhibit strong temporal dependencies (memory).

This report extends the investigation of the 3F7 short laboratory in two directions. First, we analyse **discrete stationary sources**, relaxing the i.i.d. assumption to model sources with memory. We investigate the *entropy rate* H_∞ of a stochastic process and the convergence properties of block entropies. Second, we explore **adaptive coding**, where the probability model is learned online instead of 2 separate phases (building the probability model from the complete text and encoding).

2 Overview of Short Lab

The short laboratory focused on compressing `hamlet.txt` (approx. 207kB using fixed-length ASCII encoding) using three classical techniques designed for i.i.d. sources.

Shannon-Fano Coding constructs prefix-free codes by assigning lengths $l_i = \lceil \log_2(1/p_i) \rceil$. While it guarantees unique decodability (via the Kraft inequality $\sum 2^{-l_i} \leq 1$), it is suboptimal because it ignores the combinatorial nature of packing the prefix-free code tree (codewords can still be placed at shorter leaf nodes, hence creating a more optimal encoding)

Huffman Coding is the optimal symbol-by-symbol prefix code. It builds a binary tree bottom-up by iteratively merging the two least probable symbols. Our experiments confirmed that its expected length L satisfies $H(X) \leq L < H(X) + 1$. However, its performance is constrained by the requirement that each symbol must use an integer number of bits.

Arithmetic Coding overcomes the integer constraint by encoding entire sequences as sub-intervals of $[0, 1)$. By mapping a sequence x^n to an interval of width $P(x^n)$, it achieves a code length $L \approx -\log_2 P(x^n)$ bits. For large n, the per-symbol length converges to the entropy $H(X)$. This method proved superior for longer blocks but required accurate probability models.

The limitation of all three methods in the short lab was the assumption that text is a memoryless source, ignoring the significant redundancy in English grammar and spelling.

3 Coding for Discrete Stationary Sources

3.1 Theoretical Background

We model natural language text as a sequence of random variables, specifically a *discrete stochastic process*, $\{X_i\}_{i=1}^{\infty}$. We distinguish between two primary classes of sources:

Memoryless Sources (i.i.d.): The simplest model assumes that successive symbols are i.i.d. A source is *memoryless* if the joint probability of a sequence factorizes completely into the product of its components:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i). \tag{1}$$

For such a source, the information content is additive, and the entropy of a block is simply $H(X^n) = nH(X)$. The optimal code length per symbol is thus fixed at the marginal entropy $H(X)$. The fixed-length ASCII and standard Huffman coding (without blocking) implicitly assume this model.

Stationary Sources (Sources with Memory): Natural languages like English are not memoryless; the letter 'h' is much more likely to follow 't' than 'q'. To capture this context, we relax the independence assumption while retaining time-invariance. A process is *strictly stationary* if the joint distribution of any subsequence is invariant to time shifts. That is, for any shift τ and block length n:

$$P(X_1 = x_1, \ldots, X_n = x_n) = P(X_{1+\tau} = x_1, \ldots, X_{n+\tau} = x_n). \tag{2}$$

This invariance in probability distribution directly implies the invariance of entropy. If the distribution P is shift-invariant, $H(X^n) = \mathbb{E}[-\log P(X^n)]$ must also be shift-invariant. Thus, $H(X_1, \ldots, X_n) = H(X_{1+\tau}, \ldots, X_{n+\tau})$ and conditional entropies rely solely on relative history.

Entropy Rate: For a source with memory, the marginal entropy $H(X_1)$ overestimates the true information content because it ignores the predictability provided by the context. The fundamental limit is instead given by the *entropy rate* H_∞: [1, Def 4.1]:

$$H_\infty = \lim_{n \to \infty} \frac{1}{n} H(X_1, \ldots, X_n). \tag{3}$$

This rate represents the irreducible uncertainty of the process after all past history is accounted for. To estimate H_∞, we define two sequences of block entropies:

$$H_n = \frac{1}{n} H(X_1, \ldots, X_n) \quad \text{(Entropy per symbol of } n\text{-block)} \tag{4}$$

$$H_n^* = H(X_n | X_{n-1}, \ldots, X_1) \quad \text{(Conditional entropy)} \tag{5}$$

3.2 Convergence of Entropy Rates (Theorem 1)

For stationary sources, these two sequences converge to the entropy rate H_∞ as $n \to \infty$. We provide proofs for the below properties, referencing Cover & Thomas [1, Chap 4].

Statement 1. $H_{n+1}^* \leq H_n^*$ (Conditional entropy is non-increasing).

Proof:

$$\begin{aligned} H_{n+1}^* &= H(X_{n+1} | X_n, \ldots, X_2, X_1) \\ &\leq H(X_{n+1} | X_n, \ldots, X_2) \quad \text{(Conditioning reduces entropy)} \\ &= H(X_n | X_{n-1}, \ldots, X_1) \quad \text{(By Stationarity)} \\ &= H_n^* \quad \square \end{aligned}$$

Statement 2. $H_{n+1} \leq H_n$ (Per-symbol entropy is non-increasing).

Proof: By the chain rule, $H(X_1, \ldots, X_{n+1}) = H(X_1, \ldots, X_n) + H(X_{n+1}|X_1, \ldots, X_n)$. Then

$$H_{n+1} = \frac{H(X_1, \ldots, X_{n+1})}{n+1} = \frac{nH_n + H^*_{n+1}}{n+1} \tag{6}$$

Since $H^*_{n+1} \leq H^*_n$ (from Statement 1) and $H^*_n \leq H_n$ (from Statement 3 below), then $H^*_{n+1} \leq H^*_n \leq H_n$. Here H_n is the average of joint entropy terms; adding a term H^*_{n+1} that is smaller than or equal to the previous average must reduce (or maintain) the new average. Thus $H_{n+1} \leq H_n$. □

Statement 3. $H^*_n \leq H_n$.

Proof: From the Chain Rule for entropy,

$$H(X_1, \ldots, X_n) = \sum_{i=1}^{n} H(X_i|X_{i-1}, \ldots, X_1). \tag{7}$$

Dividing by n:

$$H_n = \frac{1}{n} \sum_{i=1}^{n} H^*_i. \tag{8}$$

Since H^*_i is a non-increasing sequence (Statement 1), the current term H^*_n is less than or equal to all preceding terms H^*_i for $i < n$. Therefore, the average H_n must be greater than or equal to the minimum term H^*_n. □

Statement 4. $\lim_{n\to\infty} H_n = \lim_{n\to\infty} H^*_n = H_\infty$.

Proof: Since H^*_n is non-increasing (Statement 1) and bounded below by 0 (entropy is non-negative), the limit $\lim_{n\to\infty} H^*_n$ exists. Since H_n is the arithmetic mean (Cesàro mean) of the sequence H^*_n as $n \to \infty$, by the *Cesàro Mean Theorem* (if $a_n \to a$, then $\frac{1}{n}\sum a_i \to a$), the sequence of averages converges to the same limit [1, Thm 4.2.1]. This limit is defined as the *entropy rate* H_∞. □

3.3 Experimental Results: Entropy Rate Convergence

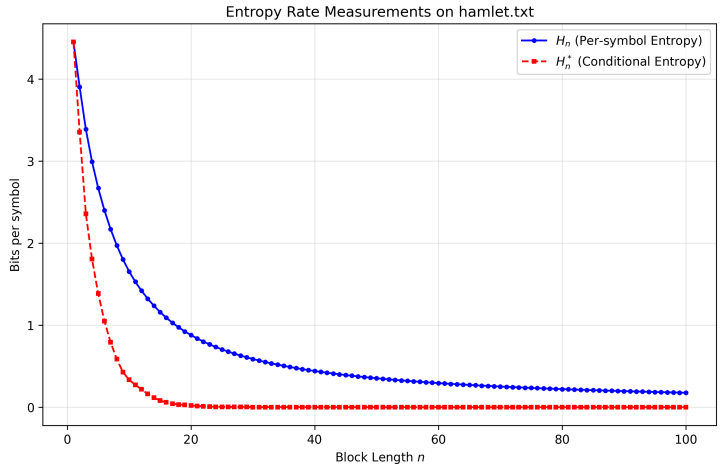

Figure 1: Entropy rate measurements on `hamlet.txt`. The estimates converge to 0 for large n due to data sparsity, not because the text is deterministic.

We measured the block entropy per symbol, $H_n = \frac{1}{n}H(X_1, \ldots, X_n)$, for $n = 1$ to 100. As shown in Fig. 1, estimates for small n ($n = 1 - 20$) decrease quickly, correctly reflecting that

English text is a source with memory; knowledge of previous characters reduces the uncertainty of the current one. However, for $n \geq 4$, the estimate continues to decay toward zero. The true entropy rate of English (H_∞) is surely not zero. Rather, this decay is a statistical artifact of an maximum likelihood estimator (of the entropy rate) operating in a high-dimensional regime with insufficient data (the "curse of dimensionality"). Accurate measurement of H_∞ is impossible without a sample size N (length of our sample `hamlet.txt`) that scales as $|\mathcal{X}|^n$ (number of possible sequences in our probability model).

3.4 Implications for Static Coding

The convergence failure observed above highlights a critical limitation for static source coding. Theoretically, to approach the entropy rate H_∞ using a static method like Huffman coding, one must group n consecutive symbols into a single "meta-symbol" (blocking). The encoder would then construct a Huffman tree for the extended alphabet $\mathcal{X}^n$, assigning codes based on the joint probabilities $P(X_1, \ldots, X_n)$.

However, in a static framework, this requires transmitting the probability model (the frequency table) to the decoder as side information. The size of this model is proportional to the alphabet size $|\mathcal{X}|^n$. As we increase n to capture longer dependencies, this overhead grows exponentially, quickly exceeding the size of the compressed data itself. Consequently, static methods cannot simply let $n \to \infty$; they are fundamentally limited to a small, finite n. To overcome this "static overhead limit" and exploit long-range memory, we must abandon the two-part code structure entirely and adopt an *adaptive* approach (discussed below in section 5)

4 Adaptive Coding

4.1 Bayesian Derivation of the Laplace Estimator

In Bayesian inference, we treat the unknown probabilities for our symbols not as fixed values, but as random variables containing uncertainty [3, p. 97]. We derive the Laplace Estimator to estimate the unknown probabilities for generating symbols given data.

4.1.1 The Beta Posterior

Consider a binary version of the problem where we simply estimate the probability θ that the next symbol is a specific character 'x'. We start with Bayes' Theorem:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)} \tag{9}$$

where D is the observed data (sequence of symbols), $P(\theta)$ is the *prior* belief, and $P(D|\theta)$ is the *likelihood* of the data.

- **Prior:** Since we assume no prior knowledge, we choose a prior that represents total ignorance. For a probability parameter $\theta \in [0, 1]$, the natural choice is the Uniform Distribution, also known as the $Beta(1, 1)$ distribution [2, Ch 3]:

$$P(\theta) = 1 \quad \text{for } 0 \leq \theta \leq 1 \tag{10}$$

- **Likelihood:** Suppose we observe a stream of length n containing n_x occurrences of 'x' and n_y occurrences of other symbols ($n_y = n - n_x$). Assuming independence of symbols given θ, the probability of seeing this specific sequence is:

$$P(D|\theta) = \theta^{n_x}(1-\theta)^{n_y}. \tag{11}$$

- **Posterior:** Multiplying the prior and likelihood gives us the posterior distribution (proportionality ignores the normalizing constant $P(D)$):

$$P(\theta|D) \propto \theta^{n_x}(1-\theta)^{n_y} \cdot 1. \tag{12}$$

This resulting distribution is a Beta distribution with parameters $(n_x + 1, n_y + 1)$. [1]As n grows, this curve becomes a sharp peak around the empirical frequency n_x/n.

4.1.2 Laplace's Rule of Succession (Binary Case)

To encode the next symbol X_{n+1}, we do not simply use the most probable value of θ (the peak). Instead, we marginalize over *all possible* values of θ. This predictive probability is equivalent to the expected value of the posterior distribution:

$$P(X_{n+1} = x|D) = \int_0^1 P(X_{n+1} = x|\theta)P(\theta|D)\, d\theta \tag{13}$$

$$= \int_0^1 \theta \cdot P(\theta|D)\, d\theta \quad \text{since } P(X_{n+1} = x|\theta) = \theta \text{ by definition} \tag{14}$$

$$= \mathbb{E}[\theta|D] \tag{15}$$

Since our posterior is a Beta distribution $Beta(\alpha, \beta)$ with $\alpha = n_x + 1$ and $\beta = n_y + 1$, we use the known formula for the mean of a Beta distribution, $\mu = \frac{\alpha}{\alpha+\beta}$ [2, 4]:

$$\hat{p}(x) = \mathbb{E}[\theta|D] = \frac{\alpha}{\alpha+\beta} = \frac{n_x+1}{(n_x+1)+(n_y+1)} = \frac{n_x+1}{n+2}. \tag{16}$$

This result is Laplace's Rule of Succession. It effectively adds 1 "pseudocount" to both outcomes, preventing zero probabilities [2, p. 52].

4.1.3 Generalization to Text (Multinomial Case)

We now generalize from binary outcomes to an alphabet of size K (e.g., $K = 256$ for extended ASCII). The unknown parameter is now a probability vector $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_K)$ where $\sum_{i=1}^K \theta_i = 1$. The natural generalization of the Beta distribution to K dimensions is the *Dirichlet distribution* [2, pg 317] with parameters $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_K)$ and probability density [2, Eq. 23.30]:

$$P(\boldsymbol{\theta}|\boldsymbol{\alpha}) = \frac{1}{B(\boldsymbol{\alpha})} \prod_{i=1}^K \theta_i^{\alpha_i - 1} \tag{17}$$

where $B(\boldsymbol{\alpha}) = \frac{\prod_i \Gamma(\alpha_i)}{\Gamma(\sum_i \alpha_i)}$ is the multivariate Beta function (normalizing constant) and each $\alpha_i > 0$ [5]. Suppose we observe a sequence with symbol counts $\mathbf{n} = (n_1, \ldots, n_K)$ where $N = \sum_i n_i$ is the total number of observations. Bayesian analysis with the uniform prior $\boldsymbol{\alpha} = (1, \ldots, 1)$ gives us the posterior:

$$P(\boldsymbol{\theta}|\mathbf{n}) = \text{Dir}(n_1 + \alpha_1, \ldots, n_K + \alpha_K) = \text{Dir}(n_1 + 1, \ldots, n_K + 1) \tag{18}$$

To predict the next symbol, we marginalize over all possible parameter values weighted by the posterior, giving us the mean of the posterior Dirichlet distribution (similar to the binary case Eq 13). For a Dirichlet distribution with parameters $(\alpha'_1, \ldots, \alpha'_K)$, the mean is [2, Eq. 23.32] $\mathbb{E}[\theta_i] = \frac{\alpha'_i}{\sum_{j=1}^K \alpha'_j}$. Substituting our posterior parameters $\alpha'_i = n_i + 1$:

$$\hat{p}(i) = \mathbb{E}[\theta_i|\mathbf{n}] = \frac{n_i+1}{\sum_{j=1}^K (n_j+1)} = \frac{n_i+1}{N+K} \tag{19}$$

[1]The general form of the Beta probability density function is: $f(\theta; \alpha, \beta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$

This is the *Laplace estimator*. It ensures every symbol has non-zero probability of at least $1/(N+K)$, which effectively adds 1 "pseudocount" to each symbol.

4.2 Experimental Performance and Analysis

We simulate adaptive arithmetic coding on `hamlet.txt`. The code length L for a sequence $x_1, \ldots, x_n$ is estimated as the sum of the log-probabilities assigned by the model at each step $L = -\sum_{i=1}^{n} \log_2 \hat{p}_{x_i}$. [2]

4.2.1 Convergence to Static Entropy

Figure 2 shows the running average rate L/n, which is the estimate of the entropy of the i.i.d distribution.

- **Initial Behaviour:** For small n, the rate is high (> 6 bits) because counts are low and the uniform prior dominates the distribution.
- **Asymptotic Behavior:** As $n \to \infty$, the estimator $\hat{p}_x$ converges to the true relative frequencies of the file. Consequently, the adaptive rate converges to the static i.i.d. entropy H_1 (≈ 4.45 bits). Note that the curve stays slightly above H_1. This redundancy represents the "learning cost" of building the model online (we haven't observed the complete data yet)

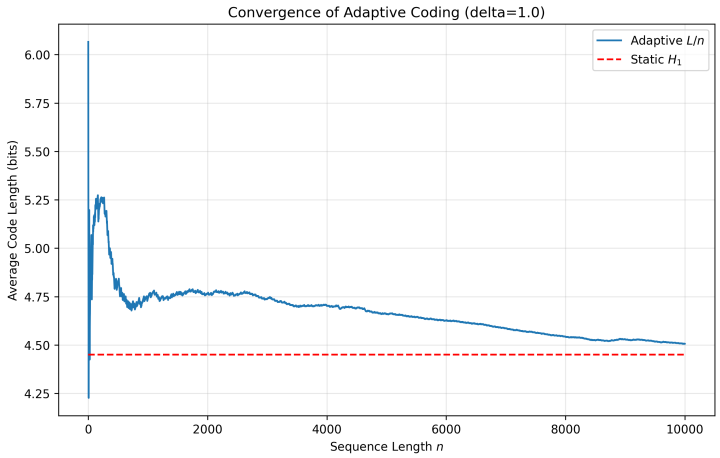

Figure 2: Convergence of 0-order Adaptive Coding (L/n) to the static entropy H_1 with $\delta = 1.0$ (Laplace Estimator) over $N = 10,000$ symbols.

4.2.2 Sensitivity to Bias Parameter δ

We investigated the impact of the pseudocount parameter δ [3] by generalizing the estimator to:

$$\hat{p}(x) = \frac{n_x + \delta}{N + K\delta} \tag{20}$$

We tested $\delta \in [0.01, 50]$ on the first $N = 10,000$ symbols of `hamlet.txt` to visualize convergence behavior (Fig. 3), and then measured the final compression rate on the *entire text* ($N = 207,039$) to determine the optimal parameter (Fig. 4).

[2] Note that the probability model here built is a still a 0-order adaptive model (we are still learning the distribution assuming each character is i.i.d.) The model is adaptive in the sense of updating its probabilities as more characters are observed

[3] Note $\delta = \alpha$ assuming a uniform prior for the Dirichlet distribution

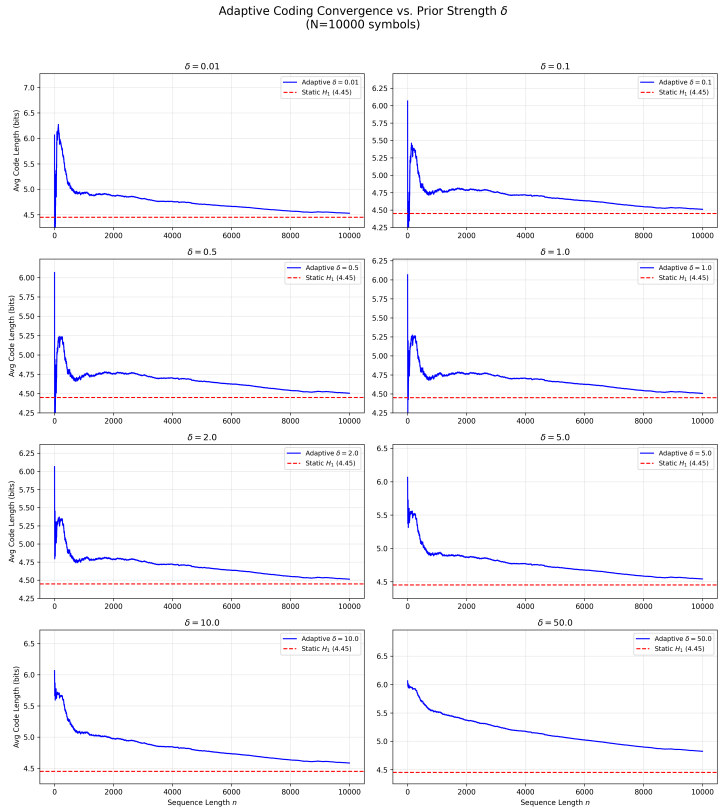

Figure 3: Adaptive coding convergence profiles for various prior strengths δ ($N = 10,000$ symbols). Small δ leads to rapid convergence but high volatility; large δ creates a stable but slow-converging model.

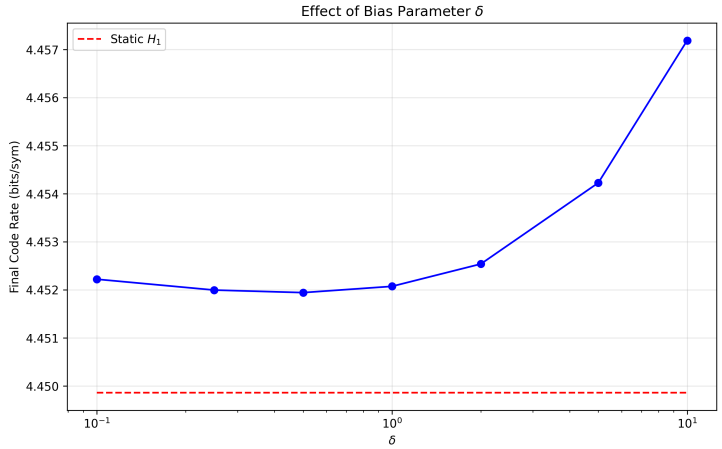

Figure 4: Effect of bias δ on the final compression rate, measured on the *entire* Hamlet text ($N \approx 207k$)

- **Small δ ($\delta \ll 1$):** The denominator is dominated by N, so the estimator approximates:

$$\hat{p}(x) \approx \frac{n_x + \delta}{N} = \frac{n_x}{N} + \frac{\delta}{N}. \tag{21}$$

  For unseen symbols ($n_x = 0$), the probability is only δ/N, yielding a harsh code length penalty of $-\log_2(\delta/N) \approx \log_2(N/\delta)$ bits. As $\delta \to 0$, this initial cost spikes dramatically, as seen in the sharp initial peaks in Fig. 3. However, once a symbol is seen, the model adapts very quickly as the estimator closely tracks the empirical frequency n_x/N
- **Large δ ($\delta \gg 1$):** The denominator is dominated by $K\delta$, so the estimator approximates:

$$\hat{p}(x) \approx \frac{n_x + \delta}{K\delta} = \frac{n_x}{K\delta} + \frac{1}{K}. \tag{22}$$

  Initially the uniform prior $1/K$ dominates, and observed counts n_x have diluted influence. As the data requires large n_x values to overcome the strong prior belief, this results in a very stable curve that converges slowly to the true entropy (Fig. 3, bottom right).
- **Optimal $\delta \approx 0.5$:** Our experiments on the full text (Fig. 4) show that $\delta = 0.5$ yields the minimum code rate, slightly outperforming the Laplace estimator ($\delta = 1$)

4.3 Comparison of Adaptive vs. Static Coding

To rigorously compare the performance of Adaptive and Static coding (instead of considering fixed offline storage), we invoke the *Minimum Description Length (MDL)* principle. [4] This principle states that the total description length $L(x_1 \dots x_n) = L(x^n)$ is composed of the cost to encode the data given a model, plus the cost of specifying the probability model itself: [1, Chap 14]

$$L(x^n) = L(\text{Data} \mid \text{Model}) + L(\text{Model}) \tag{23}$$

In our experiment, we compare the cumulative code lengths of the two approaches as a function of the sequence length n:

Static Coding (Theoretical Bound): Since static coding requires a pre-computed frequency table, it is impossible to run "online" in the same manner as adaptive coding. Instead, we plot the theoretical lower bound at each length n, calculated using the MDL penalty [2, Ch. 28]: [5]

$$L_{\text{static}}(n) \approx \underbrace{nH_n}_{\text{Data Cost}} + \underbrace{\frac{K-1}{2}\log_2 n}_{\text{Model Cost}} \tag{24}$$

Adaptive Coding (Actual Cost): This curve represents the number of bits generated by the simulated arithmetic coder up to symbol n. Unlike the static case, there is no explicit model cost ($L(\text{Model}) = 0$). Instead, the cost is the sum of the negative log-probabilities assigned by the sequential estimator:

$$L_{\text{adaptive}}(n) = \sum_{i=1}^{n} -\log_2 \hat{P}(x_i \mid x^{i-1}) \tag{25}$$

4.3.1 Experimental Results

Figure 5 presents the comparison of these two quantities on the `hamlet.txt` sequence.

[4] Interestingly enough, I actually wrote a paper using the Bayesian Information Criterion *(MDL in nats instead of bits)*, so I'm delighted to see its use here. See https://arxiv.org/abs/2409.04913

[5] The term $\frac{K-1}{2}\log_2 n$ represents the parametric complexity, the cost of specifying $K-1$ independent probability parameters to a precision of $1/\sqrt{n}$

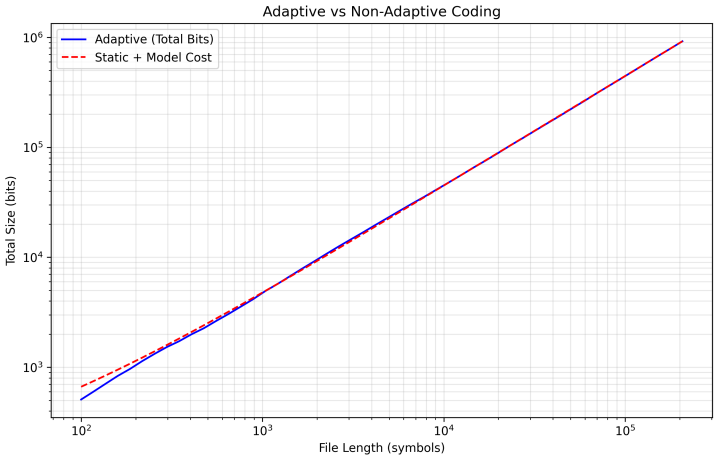

Figure 5: Comparison of Adaptive Coding (actual cumulative bits) versus Static Coding (theoretical data + MDL model cost). The curves run parallel for large n, confirming that the adaptive learning penalty scales identically to the static model overhead.

- **Short Sequences ($n < 10^4$):** Adaptive coding is superior. The static method suffers from the "overhead problem," where the fixed cost of the frequency table ($\approx \frac{K}{2} \log n$) outweighs the data payload.
- **Convergence ($n \to \infty$):** As n increases, the two curves converge. The static overhead is amortized over more symbols, while the adaptive estimator converges to the true frequencies.

5 Adaptive Coding with Memory

The previous experiments assumed a memoryless source. To approach the true entropy of English, we must estimate conditional probabilities $P(X_t \mid X_{t-1}, \ldots, X_{t-k})$. However, extending the adaptive estimator to high-order contexts k causes the "curse of dimensionality". As k increases, the number of possible contexts grows exponentially $|\mathcal{X}|^k$ despite a fixed text length N. The data becomes relatively sparse.

- **Low Order ($k = 1$):** Contexts are frequent. The counts are high, so $\hat{P}$ reflects the true language statistics and converge stably
- **High Order ($k = 5$):** The number of contexts (67^5) exceeds the file size. Most contexts are seen for the first time, where $\hat{P}$ is initially volatile

This illustrates a Bias-Variance Trade-off: short contexts have high bias (ignore contextual structure of text) but low variance (stable statistics), while long contexts capture structure but suffer from high variance due to insufficient data. Practical solutions (e.g., Prediction by Partial Matching) address this by dynamically backing off to shorter contexts when data is sparse.

6 Conclusion

The entropy rate H_∞ defines the theoretical compression limit for stationary sources, yet approaching it is practically constrained by the "curse of dimensionality." We demonstrated that adaptive coding addresses the overhead of transmitting static probability models. Ultimately, exploiting source memory faces a fundamental trade-off: increasing context depth reduces theoretical uncertainty but exponentially dilutes statistical evidence. Optimal compression, therefore, cannot simply maximize context length but must balance model complexity against the data sparsity inherent in finite streams.

References

[1] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.

[2] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[3] S. Godsill, *Estimation Theory and Inference Methods*, Lecture Notes (3F3), University of Cambridge, 2023

[4] J. Weisberg, "Laplace's Rule of Succession," *Formal Epistemology*, 2019. Available: https://jonathanweisberg.org/post/inductive-logic-2/

[5] A. Tsun, "The Beta and Dirichlet Distributions," Chapter 7.4 in *Probability & Statistics with Applications to Computing*, CS109 Course Reader, Stanford University, 2025. Available: https://web.stanford.edu/class/archive/cs/cs109/cs109.1218/files/student_drive/7.4.pdf

[6] E. P. Xing, "Bayesian Nonparametrics: Dirichlet Processes," Lecture 19 in *10-708: Probabilistic Graphical Models*, Lecture notes by C. Malings and J. Gao, Carnegie Mellon University, Spring 2014. Available: https://www.cs.cmu.edu/~epxing/Class/10708-14/scribe_notes/scribe_note_lecture19.pdf